

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3071 – Diseño de Lenguajes de Programación

Sección 30

MSc. Pablo Andrés Barreno Koch



Excelencia que trasciende

DEL VALLE
GRUPO EDUCATIVO

RECUPERACIÓN

Daniela Ramírez de León - 23053

Ciudad de Guatemala, Guatemala, 14 de mayo de 2026

1. Arquitectura del Sistema

El sistema implementa un ecosistema completo de análisis léxico y sintáctico organizado en tres capas principales. La primera capa, YALex, se encarga del análisis léxico. La segunda capa, YAPar, maneja el análisis sintáctico con soporte para múltiples métodos. La tercera capa agrupa los parsers específicos de cada lenguaje soportado, además de la interfaz gráfica web.

1.1. Capa de análisis léxico (YALex)

YALex lee archivos .yalex y construye un scanner automático mediante el siguiente pipeline:

- lexer.py — Lee y parsea el archivo .yalex extrayendo definiciones y reglas
- regex_engine.py — Convierte cada patrón regex a un AFN usando el algoritmo de Thompson
- regex_engine.py — Convierte cada AFN a un AFD usando construcción de subconjuntos
- scanner.py — Usa los AFDs para tokenizar código fuente con estrategia maximal munch

1.2. Capa de análisis sintáctico (YAPar)

YAPar toma los tokens producidos por YALex y verifica que sigan la gramática definida en el archivo .yapar. Implementa tres métodos de análisis:

Modulo	Metodo	Base teorica	Caracteristica clave
ll1_table.py	LL(1)	FIRST y FOLLOW	Descendente, sin recursion izquierda
slr_table.py	SLR(1)	LR(0) + FOLLOW	Ascendente, lookahead global
lalr_table.py	LALR	LR(1) fusionado	Ascendente, lookahead por estado

1.3. Capa de Parsers específicos

Cada lenguaje soportado tiene su propio parser que combina un tokenizador especializado con la infraestructura YAPar:

- maya_parser.py — Parser Maya Yucateco con traducción al español
- valorant_parser.py — Parser ValorantScript con tokenizador de keywords
- cow_parser.py — Parser COW para las 12 instrucciones esotericas

- `messi_parser.py` — Parser MessiScript para el lenguaje narrativo de futbol

1.4. Interfaz Web (GUI)

La interfaz gráfica es una aplicación web de una sola página servida por Flask. El backend expone endpoints REST que el frontend consume vía fetch API. Las visualizaciones del autómata y el árbol sintáctico usan D3.js.

Archivo	Responsabilidad
<code>app.py</code>	Backend Flask — endpoints REST para todos los modulos
<code>index.html</code>	Estructura HTML — 9 tabs con todos los paneles
<code>styles.css</code>	Estilos IDE oscuro con variables CSS
<code>app.js</code>	Logica del frontend — estado, API calls, renderizado
<code>viz.js</code>	Visualizaciones D3 — arbol sintactico y automata

2. Decisiones de diseño

2.1. Separación de responsabilidades

Cada archivo tiene una única responsabilidad. El lector YALex solo parsea el archivo de texto. El motor de regex solo construye autómatas. El scanner solo tokeniza. Esta separación facilita pruebas unitarias independientes y hace el código más comprensible.

2.2. Contadores de estado globales

Los estados del autómata usan contadores de clase (LR0State._counter, DFASState._counter) que se acumulan entre solicitudes al servidor Flask. Esto significa que el estado inicial de la tabla puede no ser 0 después de la primera solicitud. La solución adoptada fue hacer que el parser busque el estado inicial como `min(action_table.keys())`, ya que el estado inicial siempre se construye primero y tiene el ID más bajo.

2.3. Filtrado de tokens especiales

El scanner agrega un token EOF al final de la tokenización. Este token no debe llegar al parser porque la tabla de análisis no tiene acciones definidas para él. De manera similar, los espacios ignorados pueden producir tokens con tipo vacío (") si la regla de whitespace no produce None. Ambos casos se filtran en el backend antes de invocar al parser.

2.4. LALR sobre SLR para parsers de lenguajes

Los parsers de Maya Yucateco y ValorantScript usan LALR en vez de SLR. LALR calcula lookaheads específicos por estado (`item.lookahead`) en lugar de usar `FOLLOW(A)` global. Esto reduce conflictos falsos, evitando situaciones donde SLR reportaría un conflicto que en realidad no existe para la gramática dada porque nunca ocurre en ese estado específico.

2.5. Dos gramáticas para LL(1)

Se mantienen dos versiones del archivo de gramática de ejemplo: `sample.yapar` con recursión izquierda (compatible con SLR y LALR) y `sample_ll1.yapar` sin recursión izquierda (compatible con LL(1)). LL(1) no puede manejar recursión izquierda porque entra en un loop infinito al expandir el mismo no terminal repetidamente.

3. Análisis Paralelo para Conflictos

Los conflictos shift/reduce y reduce/reduce ocurren cuando la tabla de análisis tiene más de una acción posible para un estado y token dados. En vez de resolver el conflicto arbitrariamente (como hace YACC eligiendo siempre shift), el sistema explora todos los caminos posibles en paralelo usando hilos de Python.

3.1. Detección de conflictos

Durante la construcción de la tabla SLR o LALR, cuando se intenta agregar una segunda acción a una celda que ya tiene una acción, se registra el conflicto y la celda pasa a contener una lista de acciones en vez de una sola acción. El parser detecta esto al consultar la tabla.

3.2. Mecanismo de paralelismo

Cuando el parser encuentra una celda con lista de acciones, lanza un hilo por cada acción usando `threading.Thread`. Cada hilo recibe una copia independiente del estado del parser (pila, posición, historial de pasos) para que no interfieran entre sí. El hilo principal espera a todos los hilos con `join()` antes de continuar.

3.3. Interpretación de resultados

Al final del análisis, se tienen tantos resultados como caminos se exploraron. Si al menos uno acepta, la entrada es válida. Si todos rechazan, hay un error sintáctico. Si varios aceptan, la gramática es ambigua para esa entrada. La interfaz web muestra cada camino en el tab de Paralelismo.

4. Gramáticas Definidas

4.1. Gramática de expresiones aritméticas (sample)

Gramática de referencia con recursión izquierda. Soporta asignaciones y expresiones con los cuatro operadores aritméticos y paréntesis.

- programa $\rightarrow$ lista_sentencias lista_sentencias $\rightarrow$ lista_sentencias sentencia | sentencia
- sentencia $\rightarrow$ ID EQUALS expresion expresion $\rightarrow$ expresion PLUS termino | expresion MINUS termino | termino
- termino $\rightarrow$ termino TIMES factor | termino DIVIDE factor | factor
- factor $\rightarrow$ INT | FLOAT | ID | LPAREN expresion RPAREN

4.2. Gramática LL(1) sin recursión izquierda (sample_ll1)

Versión refactorizada de la gramática anterior, eliminando la recursión izquierda mediante la introducción de no terminales auxiliares.

expresion $\rightarrow$ termino expresion_p expresion_p $\rightarrow$ PLUS termino expresion_p | MINUS termino expresion_p | epsilon termino $\rightarrow$ factor termino_p termino_p $\rightarrow$ TIMES factor termino_p | DIVIDE factor termino_p | epsilon

4.3. Gramática Maya Yucateco

Gramática para el idioma maya yucateco (no kaqchikel ni quiche). Soporta la estructura VSO (Verbo-Sujeto-Objeto) para preguntas y SVO para oraciones. Incluye traducción al español.

consulta $\rightarrow$ pregunta INTERROGACION | oracion PUNTO | oracion pregunta $\rightarrow$ pal_interrogativa sintagma_verbal | pal_interrogativa sintagma_nominal sintagma_verbal
oracion $\rightarrow$ sintagma_nominal sintagma_verbal sintagma_nominal $\rightarrow$ LE sustantivo SUFIJO_O
| pronombre sustantivo | sustantivo sintagma_verbal $\rightarrow$ verbo | verbo preposicion sintagma_nominal

4.4. Gramática ValorantScript

Gramática para el lenguaje de programación creativo. Soporta declaración de variables, asignación, condicionales if/else y bucles while con keywords temáticas de Valorant.

programa $\rightarrow$ lista_sentencias sentencia $\rightarrow$ declaracion | asignacion | condicional | bucle | imprimir declaracion $\rightarrow$ ABILITY CREDITS ID PLANT expresion condicional $\rightarrow$ SPIKE LPAREN condicion RPAREN GG lista_sentencias GG | SPIKE LPAREN condicion RPAREN

GG lista_sentencias DEFUSE GG ... bucle $\rightarrow$ ROUND LPAREN condicion RPAREN GG
lista_sentencias GG

4.5. Gramática COW

Gramática para el lenguaje esotérico COW. Un programa valido es cualquier secuencia no vacía de las 12 instrucciones oficiales, que son case-sensitive.

4.6. Gramática MessiScript

Gramática para el lenguaje narrativo de futbol MessiScript. Un programa valido debe comenzar con 'la agarra messi' y terminar con 'le pega messiiii... gol!'. Los comandos intermedios son frases completas separadas por puntos.

programa $\rightarrow$ CMD_INICIO cuerpo CMD_FIN cuerpo $\rightarrow$ cuerpo comando | comando comando
 $\rightarrow$ CMD_VA CONTENIDO | CMD_VA | CMD_DERECHA | CMD_IZQUIERDA |
CMD_PISA | CMD_JUEGA | CMD_SIEMPRE | CMD_SIGUE | CMD_VUELVE ...

5. ValorantScript

ValorantScript surge de la idea de crear un lenguaje de programación donde la sintaxis refleje la temática de uno de mis videojuegos favoritos. El objetivo es demostrar que los parsers generados por YALex y YAPar son genéricos: pueden validar cualquier lenguaje con una gramática bien definida, incluso uno completamente inventado.

5.1. Keywords y su equivalencia

Keyword ValorantScript	Equivalente en Python	Inspiración en Valorant
ability credits	var (tipo entero)	Los credits compran habilidades
plant	= (asignacion)	Plantar la spike
spike ... gg	if ... end	La spike se activa si la condición se cumple
defuse ... gg	else ... end	Defusar la spike es el camino alternativo
round ... gg	while ... end	Cada ronda repite el ciclo
callout	print	Los callouts comunican info al equipo
gg	} (cierre de bloque)	GG = Good Game, fin
clutch	true	Cluthear es ganar en desventaja
whiff	false	Whiff = fallar el disparo

5.3. Características del lenguaje

- Tipado estático simple: solo tipo 'credits' (entero)
- Bloques delimitados por 'gg' en vez de llaves o indentación
- Comentarios con // hasta fin de línea
- Operadores aritméticos: + - * /
- Operadores de comparación: > < =
- Identificadores: letras, dígitos y guion bajo
- Números enteros positivos

5.4. Ejemplos de programas

Hola Valorant — declaracion y callout basico:

```
// Mi primer programa ValorantScript ability credits kills plant 0 callout kills
```

Condicional spike/defuse:

```
ability credits score plant 75 spike ( score > 50 ) gg callout score defuse gg callout 0 gg
```

Bucle round (while):

```
ability credits round_num plant 5 round ( round_num > 0 ) gg callout round_num  
round_num plant round_num - 1 gg
```

Conclusiones

- La transformación $\text{Regex} \rightarrow \text{AFN} \rightarrow \text{AFD}$ funciona correctamente, ya que los autómatas generados reconocen el mismo lenguaje que las expresiones regulares originales.
- El scanner construido tokeniza correctamente código fuente real, validando el funcionamiento del análisis léxico.
- LALR demostró ser más preciso que SLR, eliminando conflictos que SLR reportaba en gramáticas reales.
- El uso de paralelismo permite manejar gramáticas ambiguas explorando múltiples caminos de análisis sin rechazar la entrada.
- La generación automática de parsers confirma la flexibilidad del motor, que puede adaptarse a distintos lenguajes modificando únicamente los archivos de especificación.
- Los contadores globales de estado constituyen una limitación de diseño que debería mejorarse en futuras versiones para garantizar mayor robustez.